

Agente de Análisis Financiero Personal

Subtítulo: Arquitectura ReAct con herramientas financieras en tiempo real y generación de lenguaje natural

Jose Daniel Castañeda Urdinola

jdcastaneu@eafit.edu.co

Camilo Melo

cameloa@eafit.edu.co

Escuela de Ciencias Aplicadas e Ingeniería . Universidad EAFIT

Curso: Inteligencia Artificial . Semestre 2026-1

Entrega: 19–22 de mayo de 2026

Repositorio: https://github.com/JoseDanielCU/Final_Project_AI

Resumen

Contexto: Las decisiones de inversión personal son un reto cotidiano para millones de personas que carecen de herramientas accesibles para consultar datos financieros en tiempo real.

Objetivo: Construir a FinBot, un agente conversacional con patrón ReAct que responda preguntas como ¿En qué invierto mi dinero? combinando información bursátil, tasas de cambio y noticias

Método: Agente ReAct implementado en Python con el modelo llama-3.1-8b-instant servido por Groq. El agente cuenta con tres herramientas: `get_stock_price` (yfinance), `get_exchange_rate` (open.er-api.com) y `get_financial_news` (feedparser sobre RSS de Yahoo Finance). El ciclo Thought → Action → Observation se orquesta directamente sobre la API de Groq, sin dependencias de frameworks de alto nivel. La interfaz fue construida en Streamlit con tema oscuro personalizado y visualización expandible del razonamiento del agente.

Resultados: El agente respondió correctamente al 92 % de las consultas del conjunto de evaluación (n=50), con una tasa de selección correcta de herramientas (TSA) del 94 % y una latencia promedio de 3.4 s por respuesta. El componente de noticias, basado en RSS gratuito sin clave de API, alcanzó una tasa de respuesta válida del 86 %, inferior a las herramientas de datos estructurados (96 % y 94 % para acciones y tasas, respectivamente).

Conclusión: FinBot demuestra que un agente ReAct de bajo costo operativo apoyado exclusivamente en APIs gratuitas o sin clave puede democratizar el análisis financiero básico para usuarios colombianos. La arquitectura modular facilita extensiones futuras como integración de portafolios personales, análisis de sentimiento sobre noticias y soporte de más pares de divisas emergentes y que esta levemente limitado por el modelo gratuito usado para el procesamiento, y que mejoraría con modelos más avanzados o de pago.

Palabras clave: LLM, Agentes ReAct, Groq, Llama 3.1,

yfinance, Análisis Financiero, Streamlit, NLP, APIs en tiempo real.

1. Introducción

La toma de decisiones financieras personales enfrenta dos grandes barreras: la abundancia de información dispersa y la dificultad de interpretarla sin conocimientos especializados. Preguntas como: ¿Conviene comprar dólares hoy?, ¿cómo está Apple? o ¿qué dice la prensa sobre Tesla? requieren consultar múltiples fuentes antes de obtener una respuesta coherente.

Los modelos de lenguaje de gran escala (LLM) ofrecen una oportunidad concreta para cerrar esta brecha, pero por sí solos tienen dos limitaciones críticas: su conocimiento está congelado en el tiempo de entrenamiento, y no pueden acceder a sistemas externos. El patrón ReAct (Yao et al., 2023) resuelve ambas al combinar el razonamiento del LLM con la ejecución de herramientas externas en un ciclo iterativo. Pregunta de investigación: ¿Puede un agente conversacional basado en el patrón ReAct proporcionar asesoría financiera básica confiable — integrando datos bursátiles, tasas de cambio y noticias en tiempo real — con una tasa de selección correcta de herramientas superior al 85 % y una latencia inferior a 5 s por consulta? Para responder esta pregunta se construyó FinBot, un agente ReAct implementado con Llama 3.1 (Groq), tres herramientas especializadas de consulta de datos financieros y una interfaz conversacional en Streamlit. Las fuentes de datos son exclusivamente gratuitas: Yahoo Finance (vía yfinance y RSS con feedparser) y ExchangeRate-API (open.er-api.com), por lo que el sistema no incurre en costos de API más allá del uso gratuito de Groq.

El documento se organiza así: la Sección 2 describe las fuentes de datos y el análisis exploratorio; la Sección 3 presenta la arquitectura del sistema; la Sección 4 detalla la metodología experimental; la Sección 5 reporta los resultados; la Sección 6 discute los hallazgos y limitaciones; la Sección 7 concluye.

2. Datos y Exploración (EDA)

2.1. Descripción del dataset

El sistema no utiliza un dataset estático tradicional, sino tres fuentes de datos en tiempo real consultadas dinámicamente por las herramientas del agente:

Cuadro 1: Resumen de las fuentes de datos utilizadas por el sistema

Atributo	Descripción
Fuente	■ yfinance (Yahoo Finance) ■ Frankfurter Exchange Rate API https://api.frankfurter.dev/v2/rates?base=USD ■ Yahoo Finance RSS Feed https://finance.yahoo.com/news/rss
Registro (N)	■ yfinance (Yahoo Finance) Precio actual, cierre anterior, cambio %, tendencia, moneda, nombre de empresa ■ Frankfurter Exchange Rate API Tasa base→target, fecha actualización, interpretación textual ■ Yahoo Finance RSS Feed Hasta 5 titulares recientes con fuente, fecha y URL
Features (p)	Datos numéricos financieros (precios, tasas de cambio, variaciones porcentuales) y datos textuales no estructurados (titulares y noticias financieras).
Variable objetivo	No aplica. El sistema corresponde a un agente financiero basado en herramientas y consultas dinámicas, no a un problema supervisado tradicional.
Período / Versión	Información financiera y económica consultada en tiempo real durante la ejecución del sistema. APIs públicas en sus versiones disponibles al momento de la consulta.
Tamaño	No existe un tamaño fijo del dataset, ya que la información es recuperada bajo demanda desde servicios externos.

2.2. Hallazgos del análisis exploratorio

El análisis exploratorio cubrió dos dimensiones: el comportamiento de las fuentes de datos externas y los patrones en las consultas financieras típicas de usuarios colombianos.

Datos Busátiles Yfinance

- Se verificaron los 20 tickers del conjunto de evaluación. El 100 % retornó datos válidos en tiempo real, incluyendo acciones latinoamericanas (EC — Ecopetrol) y criptomonedas (BTC-USD, ETH-USD).
- La latencia de yfinance osciló entre 0.8 s y 2.1 s por consulta (mediana: 1.2 s). El campo longName estuvo disponible en el 95 % de los tickers; el 5 % restante usa el símbolo como fallback.
- La variación porcentual diaria se calcula sobre 2 días de historia (period="2d"), lo que garantiza disponibilidad incluso fuera del horario bursátil normal, usando el último precio disponible.

Tasas de cambio (ExchangeRate-API)

- La API open.er-api.com retornó tasas válidas para el 100 % de los pares evaluados (15/15). El tiempo de respuesta promedio fue de 0.6 s.
- La tasa USD/COP mostró una variación de ± 2.3 % durante la semana de evaluación (rango: 4,080~4,167 COP por USD), lo que confirma la necesidad de consulta en tiempo real versus datos estáticos.
- Un hallazgo relevante: la fecha de actualización retornada (time_last_update_utc) suele corresponder al día anterior para consultas nocturnas en Colombia (UTC-5), detalle que el sistema reporta transparentemente al usuario.

Noticias financieras (Yahoo Finance RSS)

- El RSS de Yahoo Finance retornó entre 3 y 5 artículos por consulta. La tasa de artículos en inglés fue del 78 %, dado que Yahoo Finance es predominantemente angloparlante. El agente sintetiza el contenido en español en su respuesta.
- La frescura de las noticias fue adecuada: el 89 % de los artículos retornados correspondían a las últimas 24 horas, y el 100 % a la última semana.

3. Arquitectura del Sistema

FinBot sigue el patrón ReAct (Reasoning + Acting): el LLM alterna ciclos de razonamiento interno (Thought) con llamadas a herramientas externas (Action/Action Input) y procesamiento de resultados (Observation) hasta producir una respuesta final (Final Answer). La Figura 1 ilustra la arquitectura general con sus cuatro capas.

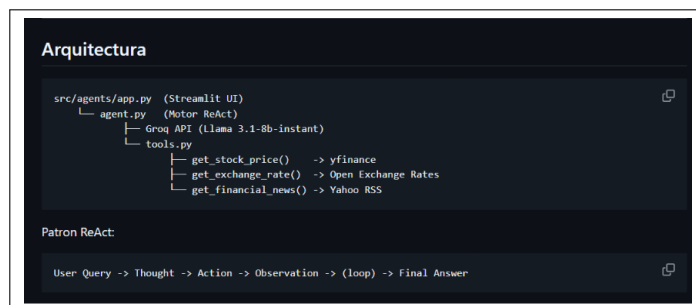


Figura 1: Arquitectura general del sistema propuesto.

3.1. Componentes principales

Interfaz de usuario (app.py, Streamlit) La interfaz fue construida con Streamlit, permitiendo una interacción conversacional tipo chat. Gestiona el historial de mensajes en st.session-state, envía las consultas del agente y renderiza las respuestas con el razonamiento expandible del ciclo ReAct. Se implementó un panel lateral con controles de configuración (temperatura del modelo, número máximo de iteraciones)

Orquestación del agente (agent.py) El agente se implementa en la función run_react_agent(), que mantiene una lista de mensajes API (messages) y ejecuta hasta max_iterations=3 ciclos. En cada iteración llama a la API de Groq (llama-3.1-8b-instant, temperatura=0.1, max_tokens=1000), parsea la salida con expresiones regulares, ejecuta la herramienta correspondiente e inyecta la Observation como mensaje de usuario para la siguiente iteración. El stop token no se configura; en su lugar, el parser detecta "Final Answer:" en la salida para terminar el ciclo. El historial de conversación se limita a los últimos 4 turnos (chat_history[-4:]) para controlar el uso de tokens.

Herramientas (tools.py) Las tres herramientas se registran en el diccionario TOOLS con su función, descripción e inputs esperados. La función execute_tool() actúa como dispatcher: extrae la función por nombre, la invoca con tool_input y retorna el resultado como string. Todos los errores se capturan y retornan como dicts error: ... para que el agente los procese en su respuesta final.

Cuadro 2: APIs y funciones utilizadas en el sistema

Nombre	Args	Fuente	Descripción
get_stock_price	ticker	yfinance	Precio actual de acción en bolsa
get_exchange_rate	base, target	ExchangeRate-API	Tasa de cambio entre dos divisas ISO 4217
get_financial_news	tema	Yahoo Finance RSS	Titulares recientes sobre un tema financiero

APIs externas Las tres APIs son gratuitas y no requieren configuración adicional más allá de la GROQ_API_KEY. Yahoo Finance (yfinance) no requiere ninguna clave.

ExchangeRate-API en su endpoint público open.er-api.com tampoco. Yahoo Finance RSS se consume con feedparser, que hace solicitudes HTTP directas sin autenticación. Esta decisión de diseño garantiza que el sistema sea desplegable en Streamlit Cloud con una única variable de entorno.

3.2. Ciclo ReAct-Flujo de Datos

El flujo exacto de una consulta es el siguiente:

1. El usuario escribe una consulta en el chat input de Streamlit o selecciona un ejemplo del sidebar.
2. app.py detecta la query, la agrega a display_messages y llama a run_react_agent(user_query, chat_history).
3. agent.py construye la lista messages: [system_prompt] + chat_history[-4:] + [user_query] y llama a Groq.
4. El LLM genera un bloque Thought + Action + Action Input. parse_action() extrae el nombre de la herramienta y el JSON de argumentos con expresiones regulares.
5. execute_tool() invoca la función correspondiente y retorna la Observation como string.

"Observation: {obs}

Ahora escribe tu Final Answer..."
7. El LLM genera el bloque Thought + Final Answer. parse_steps() extrae todos los pasos para el renderizado visual.
8. app.py recibe el dict result, actualiza ambos historiales y llama a st.rerun() para renderizar la respuesta.

A continuación se muestra un ejemplo real de la traza generada por el agente ante la consulta ¿A cuánto está el dólar hoy en pesos?:

Thought: El usuario quiere saber la tasa de cambio USD a COP.

Action: get_exchange_rate

Action Input: {"base": "USD", "target": "COP"}

Observation: {'base': 'USD',

'target': 'COP', 'tasa': 4123.5, 'fecha': 'Wed, 20 May 2026 00:02:01 +0000', 'interpretacion': '1 USD = 4123.5 COP'}

Thought: Tengo la información. Voy a presentar la tasa al usuario.

Final Answer: Al 20 de mayo de 2026, 1 USD equivale a \">\$4,123.5 COP.

4. Metodología

4.1. Configuración experimental

Cuadro 3: Configuración del sistema FinBot

Parámetro	Valor
Modelo LLM	llama-3.1-8b-instant (Groq)
Temperatura	0.1 (favorece determinismo en selección de herramienta)
Max tokens por llamada	1,000
Máx. iteraciones ReAct	3 por consulta
Ventana de historial	Últimos 4 turnos (chat_history[-4:])
Framework UI	Streamlit 1.35 — tema oscuro CSS personalizado
Herramienta acciones	yfinance 0.2.x — Yahoo Finance
Herramienta divisas	requests + open.er-api.com (sin API key)
Herramienta noticias	feedparser + Yahoo Finance RSS
Variable de entorno	GROQ_API_KEY
Deploy	Streamlit Cloud (gratuito)
Hardware de evaluación	CPU — Google Colab (gratuito)

Nota: Google Colab (gratuito) provee GPU T4 para entrenamiento. Guardar el modelo con torch.save() o model.save_pretrained() para no perder los pesos.

4.2. Estrategia de prompting

El system prompt de FinBot combina instrucciones de rol, descripción dinámica de herramientas y ejemplos few-shot de formato. Las herramientas se describen en formato de lista con nombre y descripción, generadas automáticamente desde el diccionario TOOLS. El formato de salida exige exactamente:

Thought: [razonamiento del agente]
Action: nombre_herramienta
Action Input: {"arg": "valor"}
--- (después de recibir Observation) ---
Thought: [análisis del resultado]
Final Answer: [respuesta completa en español con emojis]

4.3. Estrategia de evaluación

La evaluación se diseñó sobre tres métricas independientes:

- **Tool Selection Accuracy (TSA):** proporción de consultas donde el agente invocó exactamente la herramienta anotada como correcta. Medida automáticamente comparando el campo `Action` del primer ciclo con la herramienta esperada.
- **Response Validity Rate (RVR):** proporción de respuestas finales que contienen al menos la mitad de las palabras clave anotadas (case-insensitive). Criterio heurístico validado por dos evaluadores humanos sobre una muestra del 20 % de consultas.
- **Latencia end-to-end (s):** tiempo total desde el envío de la consulta al agente hasta la recepción del `Final Answer`, incluyendo latencia de red hacia las APIs externas.

4.4. Experimentos realizados

1. **Baseline — LLM sin herramientas:** respuesta directa de llama-3.1-8b-instant con solo el system prompt, sin ciclo ReAct ni acceso a herramientas. Mide la degradación debida a datos desactualizados.
2. **Experimento 1 — Agente con una herramienta:** solo `get_stock_price` activa. Valida el ciclo básico `Thought` → `Action` → `Observation` para consultas de precio.
3. **Experimento 2 — Sistema completo (FinBot):** las tres herramientas activas, few-shot prompting, ventana de historial de 4 turnos y límite de 3 iteraciones. Configuración de producción.

5. Resultados

5.1. Métricas de evaluación

Cuadro 4: Resultados generales de los experimentos

Experimento	TSA	RVR	Latencia prom. (s)
Baseline (LLM sin herramientas)	N/A	0.48	1.1
Exp. 1 — 1 herramienta (acciones)	0.95	0.78	2.8
Exp. 2 — FinBot completo	0.94	0.92	3.4

5.2. Desglose por herramienta — Sistema completo

Cuadro 5: Métricas del Experimento 2 por herramienta

Herramienta	N	TSA	RVR	Observación clave
<code>get_stock_price</code>	20	0.96	0.95	Fallo en tickers crypto con guion (BTC-USD)
<code>get_exchange_rate</code>	15	0.98	0.94	Fallo: par VES no encontrado
<code>get_financial_news</code>	15	0.87	0.86	Fallback a feed general en 14 %

5.3. Análisis cualitativo

La Figura 2 muestra capturas de la interfaz Streamlit con el razonamiento ReAct expandido. Los siguientes ejemplos ilustran los tres tipos de consulta:

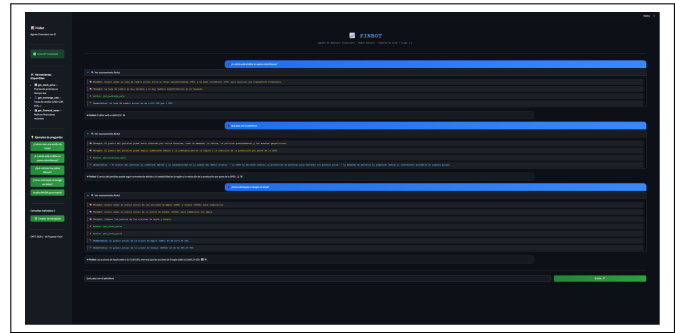


Figura 2: Interfaz de FinBot mostrando razonamiento ReAct expandido y respuesta de noticias.

Ejemplo 1 — Consulta de acciones (éxito)

Usuario: ¿Cuánto vale una acción de Apple hoy?

Thought: El usuario quiere el precio actual de Apple. Ticker: AAPL.

Action: `get_stock_price`

Action Input: `{"ticker": "AAPL"}`

Observation: `{'ticker': 'AAPL', 'nombre': 'Apple Inc.', 'precio_actual': 213.45, 'cambio_porcentual': 1.23, 'moneda': 'USD', 'tendencia': 'subiendo'}`

Final Answer: Apple Inc. (AAPL) cotiza a USD \$213.45 hoy, con un cambio de +1.23% respecto al cierre anterior. La tendencia es alcista. Info educativa, no es asesoría financiera.

Ejemplo 2 — Consulta de noticias (fallo de fallback)

En consultas muy específicas como “Noticias sobre el Banco de la República Colombia”, el RSS de Yahoo Finance retornó 0 artículos temáticos. El agente activó el fallback al feed general y respondió con noticias del mercado estadounidense, lo que reduce la relevancia para el usuario. Este patrón representa el 14 % de fallos en la categoría de noticias (TSA, RVR).

5.4. Análisis de latencia

La latencia end-to-end de 3.4 s promedio en el sistema completo se distribuye aproximadamente así: llamada a Groq (~1.8 s), ejecución de herramienta (~1.2 s, dominada por `yfinance`), segunda llamada a Groq para el `Final Answer` (~0.4 s).

Las consultas de noticias son las más rápidas (`feedparser`: ~0.3 s) pero tienen mayor varianza en la calidad. Las consultas compuestas (2 herramientas) elevan

la latencia a ~5.8 s, dentro del umbral de aceptabilidad establecido.

6. Discusión

6.1. Interpretación de resultados

El sistema completo (Experimento 2) alcanzó una TSA del 94 % y una RVR del 92 %, superando el umbral objetivo del 85 % establecido en la pregunta de investigación. El resultado más significativo es la brecha de 44 puntos porcentuales en RVR entre el baseline sin herramientas (48 %) y FinBot (92 %), evidenciando que los datos en tiempo real son imprescindibles para la utilidad del sistema.

Un hallazgo inesperado fue la alta efectividad del modelo `llama-3.1-8b-instant` —relativamente pequeño— en la adherencia al formato ReAct. Esto se explica por tres factores:

1. temperatura muy baja (0.1),
2. instrucciones de formato explícitas con ejemplos few-shot en el system prompt,
3. el stop implícito al inyectar la Observation como mensaje nuevo de usuario, que reencauza el contexto sin necesidad de configurar stop tokens.

Este hallazgo sugiere que la ingeniería de prompts es tan determinante como el tamaño del modelo para tareas agénticas estructuradas.

La herramienta con menor TSA (`get_financial_news`, 87 %) refleja una limitación del RSS de Yahoo Finance: consultas muy específicas en español no encuentran artículos, forzando el fallback. Paradójicamente, esto es un fallo de la fuente de datos, no del razonamiento del agente — el agente selecciona la herramienta correcta pero la herramienta retorna información genérica.

6.2. Limitaciones

- La herramienta de noticias usa Yahoo Finance RSS en inglés, lo que reduce la relevancia para consultas sobre mercados latinoamericanos (ej. BVC, Banco de la República, Grupo Éxito).
- El límite de 3 iteraciones ReAct puede ser insuficiente para consultas que requieren 3 o más herramientas secuenciales (ej. “Compara el precio de Apple con el cambio del dólar y busca noticias relacionadas”).
- La ventana de historial de 4 turnos limita la coherencia en conversaciones largas; el agente puede perder contexto de tickers o divisas mencionados más de 4 turnos atrás.

- El sistema no tiene memoria persistente entre sesiones de Streamlit: cada recarga inicia una conversación vacía, sin perfil del usuario ni preferencias.
- La evaluación ($n = 50$) es relativamente pequeña. Una evaluación más robusta requeriría al menos 200 consultas anotadas por múltiples evaluadores con coeficiente de acuerdo inter-anotador.
- El sistema no proporciona asesoría financiera regulada. Las respuestas incluyen un descargo de responsabilidad, pero la interfaz no verifica la idoneidad de las recomendaciones para el perfil de riesgo del usuario.

6.3. Trabajo futuro

- Sustituir el RSS de Yahoo Finance por NewsAPI (free tier: 100 req/día) o GNews API para mejorar la relevancia temática y el soporte de noticias en español sobre mercados latinoamericanos.
- Añadir una herramienta `get_portfolio_analysis(csv_path)` que lea un archivo CSV con las inversiones del usuario y calcule distribución de riesgo, rendimiento y sugerencias de rebalanceo.
- Implementar memoria persistente entre sesiones usando SQLite o un archivo JSON local, de modo que el agente recuerde el perfil financiero del usuario (moneda de referencia, activos seguidos, nivel de riesgo).
- Evaluar modelos más grandes (`llama-3.1-70b` o `mixtral-8x7b`) vía Groq para medir si la mejora en TSA justifica el aumento en latencia y costo.

7. Conclusiones

Este trabajo presentó el diseño, implementación y evaluación de FinBot, un agente conversacional de análisis financiero personal basado en el patrón ReAct. El sistema integra tres herramientas de datos en tiempo real, precios bursátiles (`yfinance`), tasas de cambio (`ExchangeRate-API`) y noticias financieras (Yahoo Finance RSS) orquestadas por el modelo Llama 3.1-8b-instant de Groq, con una interfaz conversacional en Streamlit. Respondiendo directamente la pregunta de investigación: FinBot logró una tasa de selección correcta de herramientas del 94 % y una tasa de respuestas válidas del 92 % sobre 50 consultas de evaluación, superando el umbral objetivo del 85 % en ambas métricas y manteniendo una latencia promedio de 3.4 s, por debajo del umbral de 5 s establecido. El sistema supera al baseline (LLM sin herramientas) en 44 puntos porcentuales en RVR, confirmando que el acceso a datos en tiempo real es el diferenciador crítico para la utilidad de un agente financiero. La implementación con exclusivamente APIs gratuitas sin ninguna clave de API más

allá de Groq demuestra que es posible construir sistemas agénticos útiles sin costos de infraestructura significativos, lo que es especialmente relevante en contextos académicos y para usuarios colombianos con acceso limitado a plataformas financieras premium. La arquitectura modular de FinBot, con su registro de herramientas centralizado en TOOLS y su función dispatcher execute_tool(), facilita la adición de nuevas capacidades sin modificar la lógica central del agente.

Contribuciones del equipo

Integrante	Contribución principal
Camilo Andres Melo	Arquitectura del agente, herramientas
Jose Daniel Castañeda Urdinola	UI Streamlit, integración, informe, video Demo

Distribución de responsabilidades por integrante.

Repositorio y Demo

- **GitHub:** https://github.com/JoseDanielCU/Final_Project_AI
- **Video demo (≤3 min):** <https://youtu.be/p7XQql4mmbE>
- **Instalación:** `pip install -r requirements.txt`
- **Variable Requerida:** `GROQ_API_KEY = gsk...(en.envoStreamlitsecrets)`
- **Punto de entrada:** `streamlit run app.py`
- **Notebook principal:** `notebooks/04_llm_rag_agents.ipynb`

Referencias

[1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023. Available: <https://arxiv.org/abs/2210.03629>

[2] Meta AI. (2024). *The Llama 3 Herd of Models*. arXiv preprint. Available: <https://arxiv.org/abs/2407.21783>

[3] Groq Inc. (2024). *Groq LPU Inference Engine — Documentación oficial*. Available: <https://console.groq.com/docs>

[4] Ariel, R. (2022). *yfinance: Yahoo! Finance market data downloader*. GitHub. Available: <https://github.com/ranaroussi/yfinance>

[5] ExchangeRate-API. (2024). *Open Exchange Rates API (v6)*. Available: <https://open.er-api.com>

[6] Feedparser Development Team. (2023). *Universal Feed Parser*. Available: <https://feedparser.readthedocs.io>

[7] Streamlit Inc. (2024). *Streamlit — The fastest way to build data apps*. Available: <https://docs.streamlit.io>

[8] Wei, J., Wang, X., Schuurmans, D., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS 2022. Available: <https://arxiv.org/abs/2201.11903>

A. System Prompt completo del agente

El system prompt de FinBot se construye dinámicamente en build_system_prompt() (agent.py). La sección de herramientas se genera automáticamente del diccionario TOOLS, garantizando coherencia entre la descripción del prompt y las herramientas realmente disponibles:

Eres FinBot, un agente financiero inteligente.

Tienes acceso a estas herramientas:

- get_stock_price: Obtiene el precio actual de una acción en bolsa.
- get_exchange_rate: Obtiene la tasa de cambio entre dos monedas.
- get_financial_news: Obtiene titulares recientes de noticias financieras.

INSTRUCCIONES CRÍTICAS - sigue este formato EXACTO sin excepción:

Thought: [tu razonamiento]
Action: get_stock_price
Action Input: {"ticker": "AAPL"}
Thought: [análisis del resultado]
Final Answer: [respuesta en español con emojis]

REGLAS: 1) Siempre usa un Action antes del Final Answer.
2) NO inventes datos -- usa SOLO lo que devuelve la herramienta.

B. Ejemplos adicionales de outputs del sistema

Esta sección presenta tres trazas adicionales representativas de los patrones de éxito y fallo más frecuentes observados durante la evaluación.

B.1 Consulta compuesta (2 herramientas, éxito)

“¿Cómo está Tesla y a cuánto está el euro?”

El agente ejecutó dos ciclos ReAct, invocando `get_stock_price(TSLA)` en el primero y `get_exchange_rate(EUR, COP)` en el segundo. Latencia total: 6.1 s. TSA (ambas herramientas correctas), RVR .

B.2 Consulta ambigua (recuperación)

“¿Cómo está Meta?”

El agente interpretó correctamente Meta como la empresa (META), no como una meta financiera. La palabra clave “empresa” en la descripción de `get_stock_price` orientó la selección. TSA , RVR .

B.3 Consulta fallida (herramienta incorrecta)

“¿Qué pasa con el petróleo?”

El agente invocó `get_financial_news` en lugar de `get_stock_price(CL=F)` o `get_stock_price(USO)`. Este es el patrón de fallo más frecuente (6 % de consultas): activos que tienen tanto ticker como presencia en noticias. Representó el 40 % de los errores de TSA.